

Übung 4: Java 11 HttpClient

In dieser Übung lernen wir den neuen Java 11 HttpClient kennen.

Setup

Wir benötigen für diese Aufgabe einen lokal laufenden Webserver, der Anfragen bearbeiten kann. In diesem Verzeichnis findest du eine Datei "LocalEchoServer.java", die du mit dem Java-CLI starten kannst:

```
java LocalEchoServer.java`

=====
Local Echo Server started on port 8080
Endpoints available:
GET http://localhost:8080/get
GET http://localhost:8080/status/{code}
POST http://localhost:8080/post
GET http://localhost:8080/delay/{seconds}
GET http://localhost:8080/stream/{lines}
=====
```

Aufgabe 1: HTTP GET

Ziel: Synchroner GET-Request.

1. Erstelle einen `HttpClient` (nutze `HttpClient.newHttpClient();`).
 2. Baue einen `HttpRequest`:
 - URL: `http://localhost:8080/get?status=active`
 - Methode: `GET` (Dies ist der Standard, wenn man nichts anderes angibt)
 3. Sende den Request synchron (`client.send(...)`) und nutze `BodyHandlers.ofString()` um die Antwort als Text zu erhalten.
 4. Gib den Statuscode und den Body auf der Konsole aus.
-

Aufgabe 2: HTTP POST

Ziel: POST-Request mit Headern und JSON-Body.

1. Erstelle einen JSON-String (mit oder ohne Text Block): `{"id": "DE-999", "owner": "Mustermann"}`.
 2. Baue einen `HttpRequest`:
 - URL: `http://localhost:8080/post`
 - Header: `Content-Type: application/json`
 - Header: `X-Auth-Token: secret123`
 - Methode: `POST`. Nutze `BodyPublishers.ofString(json)`.
 3. Sende den Request und prüfe, ob der Statuscode 200 ist.
-

Aufgabe 3: Timeouts

Lernziel: Konfiguration von **Timeouts** und Exception-Handling.

1. Baue einen Request an `http://localhost:8080/delay/5`. (Dieser Endpunkt wartet künstlich 5 Sekunden, bevor er antwortet).
2. Setze im `HttpRequest.newBuilder()` ein **Timeout von 2 Sekunden** (`.timeout(Duration.ofSeconds(2))`).
3. Sende den Request synchron.

4. Fange die auftretende Exception ab (Tipp: Es ist eine `HttpTimeoutException` oder allgemein `IOException`) und gebe eine Fehlermeldung aus.
-

Aufgabe 4: Async

Ziel: Asynchrone Requests (`CompletableFuture`) parallel ausführen.

Das Dashboard soll **gleichzeitig** drei Informationen laden, um Wartezeit zu sparen:

1. Status (`/get?q=status`)
2. Quota (`/get?q=quota`)
3. News (`/get?q=news`)

Aufgabe:

- Starte drei Requests mit `client.sendAsync(...)`.
 - Sammle die Ergebnisse. Nutze `CompletableFuture.allOf(...)`, um zu warten, bis *alle* fertig sind.
 - Gib erst dann “Alle Daten geladen!” aus und zeige die Ergebnisse an. Du kannst die Ergebnisse mit der `.get()`-Methode abrufen.
-

Aufgabe 5: Streaming

Ziel: Streaming, Fehlerbehandlung und Pipeline-Verarbeitung.

Aufgabe:

1. Sende einen Request an `http://localhost:8080/stream/5` (simuliert 5 Zeilen Stream).
2. Nutze `BodyHandlers.ofLines()` (Java 11+), um einen `Stream<String>` zu erhalten.
3. Verarbeite den Stream **noch während er empfangen wird**:
 - Filtere Zeilen heraus, die leer sind.
 - Transformiere jede Zeile in Großbuchstaben.
 - Gib jede Zeile sofort aus.